

->Java swing is *used to create window-based applications or desktop applications.*

->The javax.swing package provides classes : JButton, JTextField, JTextArea, JRadioButton, JCheckbox, Jmenuetc.

Steps to create GUI:

- 1.CreateinstanceofJFrame
- 2.Set size of JFrame
- 3.Set layoutmanagers// if no then set null
- 4.CreateinstanceofJbutton,JTextFieldetc..
- 5.Set position and size of a button,TextFieldetc
- 6.Add component in Frame
- 7.Set Visible True For JFrame

->**Commonly used Methods of Component class**

Method	Description
public void add(Component c)	add a component on another component.
public void setSize(int width,int height)	sets size of the component.
public void setLayout(LayoutManager m)	sets the layout manager for the component.
public void setVisible(boolean b)	sets the visibility of the component. It is by default false.

->**setBounds()**: setBounds(intx-coordinate,inty-coordinate,intwidth,intheight)

->**There are two ways to create a frame:**

- By creating the object of Frame class
- By extending Frame class (inheritance)

->Add image using JLabel

```
JLabeljl=new JLabel(new ImageIcon("bimg2.jpg"));
JPaneljp=new JPanel();
jp.add(jl);
```

->Java JButton:

The**JButton**classisusedtocreatealabeledbuttonwhichperformsomeactionwhenthebuttonispushed.Itin

heritsAbstractButtonclass.

Methods	Description
public void setText(String text)	Sets a String message on the JButton.
public String getText()	Gets a String message of JButton.
public void setIcon(Icon icon)	Sets an icon or image over the JButton.
public Icon getIcon()	Gets the icon or image of the JButton.
void setHorizontalTextPosition(int textPosition)	Sets the button message on the LEFT/RIGHT of its icon or image .
void setVerticalTextPosition(int textPosition)	Sets the button message on the TOP/BOTTOM of its icon or image.

Constructor	Description
public JButton()	Creates a button with no text on it..
public JButton(String text)	Creates a button with a text on it.
public JButton(Icon image)	Creates a button with an icon on it.
public JButton(String text, Icon image)	Creates a button with a text and an icon on it.

->Steps to perform action on button click

1. Importing Packages:

```
import javax.swing.*;  
import java.awt.*;  
import java.awt.event.*;
```

2.Create Class which Implementing ActionListener Interface: class classnameimplements
ActionListener

3. Create Button and add in Frame:

4. Registering ActionListener to the JButton:

In this step, we will add or can say register ActionListener to the JButton. For this, we have to call
addActionListener() method using the object of the JButtonclass.

5. Override actionPerformed() method

->actionPerformed() method

IfwehaveimplementedtheActionListenerinterfaceinanyclass,thenwemusthavetooverrideitsmethodw
hichisactionPerformed(ActionEvent)whichtakesaparameterActionEvent(aclassdefinedinpackagejava
.awt.event).NowwhensomeoneclicksonthebuttontheactionPerformed()methodiscalled.

->Java JLabel:

TheobjectofJLabelclassisacomponentforplacingtextinacontainer.Itisusedtodisplayasinglelineofreadon
lytext.The textcanbechangedbyanapplicationbutausercannotedititdirectly.ItinheritsJComponentclass.

Methods	Description
String getText()	t returns the text string that a label displays.
void setText(String text)	It defines the single line of text this component will display.
void setHorizontalAlignment(int alignment)	It sets the alignment of the label's contents along the X axis.
Icon getIcon()	It returns the graphic image that the label displays.
int getHorizontalAlignment()	It returns the alignment of the label's contents along the X axis.
Constructor	Description
JLabel()	Creates a JLabel instance with no image and with an empty string for the title.
JLabel(String s)	Creates a JLabel instance with the specified text.
JLabel(Icon i)	Creates a JLabel instance with the specified image.
JLabel(String s, Icon i, int horizontalAlignment)	Creates a JLabel instance with the specified text, image, and horizontal alignment.

->Java JTextField: The object of a JTextFieldclass is a text component that allows the editing of a single line text. It inherits JTextComponentclass.

JTextFieldclass declaration

—Let's see the declaration for javax.swing.JTextFieldclass.

publicclassJTextFieldextendsJTextComponentimplementsSwingConstants

Methods	Description
void addActionListener(ActionListener l)	It is used to add the specified action listener to receive action events from this textfield.
Action getAction()	It returns the currently set Action for this ActionEvent source, or null if no Action is set.
void setFont(Font f)	It is used to set the current font.
void removeActionListener(ActionListener l)	It is used to remove the specified action listener so that it no longer receives action events from this textfield.
Constructor	Description
JTextField()	Creates a new TextField
JTextField(String text)	Creates a new TextField initialized with the specified text.
JTextField(String text, int columns)	Creates a new TextField initialized with the specified text and columns.
JTextField(int columns)	Creates a new empty TextField with the specified number of columns.

->JRadioButtonclass

The JRadioButtonclass is used to create a radio button. It is used to choose one option from multiple options.

Methods	Description
<code>void setText(String s)</code>	It is used to set specified text on button.
<code>String getText()</code>	It is used to return the text of the button.
<code>void setEnabled(boolean b)</code>	It is used to enable or disable the button.
<code>void setIcon(Icon b)</code>	It is used to set the specified Icon on the button.
<code>Icon getIcon()</code>	It is used to get the Icon of the button.
<code>void setMnemonic(int a)</code>	It is used to set the mnemonic on the button.
<code>void addActionListener(ActionListener a)</code>	It is used to add the action listener to this object.

Steps:

```

JRadioButtonr1=new JRadioButton("Male");
JRadioButtonr2=new JRadioButton("Female");
r1.setBounds(75,50,100,30);
r2.setBounds(75,100,100,30);
ButtonGroupbg=new ButtonGroup();
bg.add(r1);
bg.add(r2);
f.add(r1);
f.add(r2);

```

To check which radio button has selected

```

JRadioButtonr1=new JRadioButton("Male");
if(r1.isSelected())
{
String gender="Male";
}

```

->JTextAreaclass

It represents a multi line area that displays text. It is used to edit the text .JTextAreainherits JComponent class.

Methods	Description
void setRows(int rows)	It is used to set specified number of rows.
void setColumns(int cols)	It is used to set specified number of columns.
void setFont(Font f)	It is used to set the specified font.
void insert(String s, int position)	It is used to insert the specified text on the specified position.
void append(String s)	It is used to append the given text to the end of the document.

Constructor	Description
JTextArea()	Creates a text area that displays no text initially.
JTextArea(String s)	Creates a text area that displays specified text initially.
JTextArea(int row, int column)	Creates a text area with the specified number of rows and columns that displays no text initially.
JTextArea(String s, int row, int column)	Creates a text area with the specified number of rows and columns that displays specified text.

->JComboBoxclass

JComboBox inherits JComponent class . JComboBox shows a popup menu that shows a list and the user can select a option from that specified list . JComboBox can be editable or read-only.

Methods	Description
void addItem(Object anObject)	It is used to add an item to the item list.
void removeItem(Object anObject)	It is used to delete an item to the item list.
void removeAllItems()	It is used to remove all the items from the list.
void setEditable(boolean b)	It is used to determine whether the JComboBox is editable.
void addActionListener(ActionListener a)	It is used to add the ActionListener .
void addItemListener(ItemListener i)	It is used to add the ItemListener

Constructor	Description
JComboBox()	Creates a JComboBox with a default data model.
JComboBox(Object[] items)	Creates a JComboBox that contains the elements in the specified array .
JComboBox(Vector<?> items)	Creates a JComboBox that contains the elements in the specified Vector

->Steps to perform event in JComboBox

✓Implement ItemListenerinterface

✓Register comboboxusing addItemListener()

✓Define itemStateChanged(ItemEvent)

To read item from combo box: `JComboBoxobj.getSelectedItem();`

->JTableclass

The JTableclass is used to display data in rows and columns wise.

Constructors in JTable:

•**JTable():** A table is created with empty cells.

•**JTable(int rows, int cols):** Creates a table of size rows * cols.

•**JTable(Object[][] data, Object []Column):** A table is created with the specified name where []Column defines the column names.

->JColorChooserclass

The JColorChooserclass is used to create a color chooser dialog box so that user can select any color. It inherits JComponent class.

`JColorChooser.showDialog(this,"Selecta color",initial_color_name);`

->JProgressBarclass

he JProgressBarclass is used to display the progress status of the task. It inherits JComponentclass.

Method	Description
void setStringPainted(boolean b)	It is used to determine whether string should be displayed.
void setString(String s)	It is used to set value to the progress string.
void setOrientation(int orientation)	It is used to set the orientation, it may be either vertical or horizontal by using SwingConstants.VERTICAL and SwingConstants.HORIZONTAL constants.
void setValue(int value)	It is used to set the current value on the progress bar.
Constructor	Description
JProgressBar()	It is used to create a horizontal progress bar but no string text.
JProgressBar(int min, int max)	It is used to create a horizontal progress bar with the specified minimum and maximum value.
JProgressBar(int orient)	It is used to create a progress bar with the specified orientation, it can be either Vertical or Horizontal by using SwingConstants.VERTICAL and SwingConstants.HORIZONTAL constants.
JProgressBar(int orient, int min, int max)	It is used to create a progress bar with the specified orientation, minimum and maximum value.

->JSliderclass

TheJavaJSliderclassisusedtocreatethesliderinGUI.ByusingSliderwecanselectavaluefromgivenrange.

->common constructors used in JSliderare:

JSlider(): creates a slider with default initial value 50 and range 0-100.

JSlider(int operation): develops a slider with the specified orientation set by the user.

JSlider(int min, int max, int values): creates a horizontal slider using the given min and max.

JSlider(int min, int max): develops a horizontal slider with specified min, max, and value.

JSlider(int orientation, int min, int max, int value): develop a slider with specified orientation, that must be either JSlider.HORIZONTALor JSlider.VERTICAL.

->Methods:

void setMinorTickSpacing(int p): sets the minor tick spacing in the slider.

void SetMajorTickSpacing(int p): sets the major tick spacing.

void setMinimum(int p): sets the minimum value of the slider.

void setMaximum(int p): sets the maximum value of the slider.

void setPaintsTicks(booleanbl): determines that tick mark is painted.

void setPaintLabels(booleanbl): tests whether labels are painted.

void setPaintTracks(booleanbl): determines whether the track is painted.

-

>Thesmallverticallinesthatcalibrateasliderarecalled**tickmarks**.Thelonger,thickermarksaremajorticksa ndthethinnermarksareminorticks.Themajortickscanbelabeled.

LayoutManagers

✓LayoutManageris an interface that is used to arrange components in a specific manner.

✓It is implemented by all the classes of layout managers:

➤NullLayout

➤BorderLayout

➤FlowLayout

➤GridLayout

➤GridBagLayout

➤CardLayout

➤GroupLayout

➤BoxLayout

->NullLayout

NullLayoutmeansthatnolayoutmanagerisassignedandthecomponentscanbeputatspecificx,ycoordinat es.

->BorderLayout

The border layout places components in up to five areas: center, north, south, east and west. Each area can contain only one component.

Possible values: CENTER, NORTH, SOUTH, EAST, WEST, PAGE_START, PAGE_END, LINE_START and LINE_END.

To insert horizontal and vertical gaps between buttons:

`BorderLayout(int hgap, int vgap)`

->GridLayout

The class **GridLayout** arranges the components in a rectangular grid.

Constructor	Description
GridLayout()	a grid layout with one column per component in a row
GridLayout(int rows, int columns)	a grid layout with the given rows and columns but no gaps between the components.
GridLayout(int rows, int columns, int hgap, int vgap):	a grid layout with the given rows and columns along with given horizontal and vertical gaps.

Method	Description
int getColumns()	Gets the number of columns in this layout.
int getHgap()	Gets the horizontal gap between the components.
int getRows()	Gets the number of rows in this layout.
int getVgap()	Gets the vertical gap between the components.
void setColumns(int cols)	Sets the number of columns in this layout to the specified value.
void setHgap(int hgap)	Sets the horizontal gap between the components to the specified value.
void setRows(int rows)	Sets the number of rows in this layout to the specified value.

->FlowLayout

The flow layout manager arranges components in a row from left to right, starting a new row if no more components fit into a row. Flow layouts are typically used to arrange buttons in a panel.

Note:

✓ `FlowLayout` is default layout for `JPanel`.

✓ If width of `JFrame` is less then components shifts in next row.

If width of `JFrame` is more then it aligns component to the center.

Three constant we can define in `FlowLayout`: `LEFT`, `RIGHT` and `CENTER`

->BoxLayout

The **BoxLayout** class is used to arrange the components either vertically (along Y-axis) or horizontally (along X-axis). In **BoxLayout** class, the components are put either in a single row or a single column. The components will not wrap so, for example, a horizontal arrangement of components will stay horizontally arranged when the frame is resized.

->Constructor

BoxLayout(Container c, int axis): Creates a **BoxLayout** class that arranges the components with the X-axis or Y-axis.

->Methods

addLayoutComponent(Component cmp, Object obj): It is not used by this class.

getLayoutAlignmentX(Container con): Returns the alignment along the X-axis for the container.

getLayoutAlignmentY(Container con): Returns the alignment along the Y-axis for the container.

maximumLayoutSize(Container con): Returns the maximum dimensions that the target container can use to lay out the components it contains.

minimumLayoutSize(Container con): Returns the minimum dimensions needed to lay out the components contained in the specified target container.

removeLayoutComponent(Component cmp): It is not used by this class.

layoutContainer(Container tar): Called by the AWT when the specified container needs to be laid out.

->CardLayout

The **JavaCardLayout** class manages the components in such a manner that only one component is visible at a time.

• **CardLayout()**: creates a card layout with zero horizontal and vertical gap.

• **CardLayout(int hgap, int vgap)**: creates a card layout with the given horizontal and vertical gap.

->Methods

• **public void next(Container parent)**: is used to flip to the next card of the given container.

- public void previous(Container parent):**is used to flip to the previous card of the given container.
- public void first(Container parent):**is used to flip to the first card of the given container.
- public void last(Container parent):**is used to flip to the last card of the given container.
- public void show(Container parent, String name):**is used to flip to the specified card with the given name.

Event Listener

It is an object which is notified when an event occurs.

Java Interfaces	Event They Listen
ActionListener	Listens for and handles button clicks
KeyListener	Listens for and handles key events
MouseListener	Listens for and handles mouse events

Classes that represent events

vents Classes	Description
ActionEvent	Generated when an action has occured (e.g- button pressed)
AdjustmentEvent	Generated when scroll bar is manipulated
ContainerEvent	Generated when container is changed (added or removed)

Event Classes and Associated Listener Interfaces

Event Classes	Associated Listener Interface
ActionEvent	ActionListener
AdjustmentEvent	AdjustmentListner
ContainerEvent	ContainerListener
FocusEvent	FocusListener
ComponentEvent	ComponentListener
ItemEvent	ItemListener
KeyEvent	KeyListener